

Bank Account Fraud Detection: Fraud Detection with Machine Learning and Deep Learning

Dataset: Bank Account Fraud (BAF) suite, NeurIPS 2022, file `Base.csv` (Kaggle `sgpjesus/bank-account-fraud-dataset-neurips-2022`) | **Tools:** Python, scikit-learn, LightGBM, Keras | **Platform:** Google Colab

Problem statement. When someone applies online for a bank account, the bank must decide in seconds whether the application is genuine or a **fraudulent account opening** (stolen identity, fake identity, money-mule account). Only about 1 in 100 applications is fraud, so checking everything by hand is impossible. The bank needs a model that gives every application a **fraud risk score** and sends the riskiest ones to an analyst first.

Business objective. Catch as much fraud as possible without flagging too many genuine customers (every false alarm delays a real customer and costs analyst time).

ML objective. Predict `fraud_bool` (1 = fraud, 0 = genuine) from the application details, the device and session behaviour and the recent "velocity" counts. Compare Logistic Regression, Random Forest and a tuned LightGBM model.

DL objective. Train a small Keras neural network (MLP) on the same data and check honestly whether it beats the machine learning models.

Dataset. 1,000,000 applications (32 columns) collected over **8 months** (`month` = 0 to 7). The file stores "missing" as `-1`, not as an empty cell, so we handle that first. `month` is used **only to split the data by time**, never as a model input. **No number in this notebook is typed in by hand: every count, rate and metric is calculated when you run the cells.**

How to run. Runtime > Run all. A full run should take roughly 20 to 40 minutes on a free Colab CPU. Set `FAST_MODE = True` in the next cell for a quick test (smaller sample, fewer trees, fewer epochs).

```
!pip install -q imbalanced-learn shap kagglehub

import os, glob, gc, time, random, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
from IPython.display import display, Markdown

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
```

```

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV, PredefinedSpl
from sklearn.inspection import permutation_importance
from sklearn.metrics import (accuracy_score, precision_score, recall_
                             confusion_matrix, roc_curve, precision_r
from imblearn.under_sampling import RandomUnderSampler      # resampl
from imblearn.over_sampling import RandomOverSampler, SMOTE
from lightgbm import LGBMClassifier
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers, callbacks

warnings.filterwarnings("ignore")
SEED = 42
FAST_MODE = False                                     # True = quick test run
SAMPLE_FRAC = 0.10 if FAST_MODE else 0.30 # share of the 1M rows use
N_TREES, N_ITER, EPOCHS = (60, 2, 3) if FAST_MODE else (300, 8, 40)
COST_FN, COST_FP = 500, 10                             # ASSUMED costs: a missed
TARGET = "fraud_bool"
TRAIN_MONTHS, VAL_MONTHS, TEST_MONTHS = range(0, 5), [5], [6, 7] #
random.seed(SEED); np.random.seed(SEED); keras.utils.set_random_seed(
sns.set_theme(style="whitegrid"); plt.rcParams.update({"figure.dpi":
pd.set_option("display.max_columns", 60); pd.set_option("display.floa
BLUE, RED = "#4C72B0", "#C44E52"; PALETTE = {0: BLUE, 1: RED}

def interpret(*lines):
    """Short data-driven interpretation shown under an output."""
    display(Markdown("**Interpretation**\n\n" + "\n".join(f"- {l}" fo

```

✓ Phase 1: Data Understanding, Cleaning and EDA

We first load the data and check what is really in the file, because nothing is assumed. We then clean it, and explore it with charts to see what fraudulent applications look like.

```

def find_base_csv():
    """Download with kagglehub; if Kaggle cannot be reached, ask for
    try:
        import kagglehub
        folder = kagglehub.dataset_download("sgpjesus/bank-account-fr
        hits = glob.glob(os.path.join(folder, "**", "Base.csv"), recu
        if hits: return hits[0]
    except Exception as err:
        print("kagglehub failed:", repr(err))
    from google.colab import files
    return [k for k in files.upload() if k.lower().endswith(".csv")]

raw_df = pd.read_csv(find_base_csv())

```

```
N_RAW = len(raw_df)
df = raw_df.sample(frac=SAMPLE_FRAC, random_state=SEED).reset_index(drop=True)
del raw_df; gc.collect()
print(f"Full file: {N_RAW:,} rows | working sample: {len(df):,} rows |")
display(df.head(3)); df.info()

# Decide the role of every column from the real file
assert TARGET in df.columns and "month" in df.columns, f"Expected columns: {TARGET}, {SENTINEL}"
SENTINEL = [c for c in ["prev_address_months_count", "current_address_months_count", "session_length_in_minutes", "device_distinctiveness"] if c in df.columns]
CAT_RAW = [c for c in df.columns if not pd.api.types.is_numeric_dtype(df[c])]
NUM_RAW = [c for c in df.columns if c not in CAT_RAW + [TARGET, "month"]]
BINARY_RAW = [c for c in NUM_RAW if df[c].nunique() == 2]
interpret(f"Target: **{TARGET}**. Categories: {CAT_RAW}. Binary flags: {BINARY_RAW}. Sentinel: {SENTINEL}. Columns where `-1` means 'missing': {SENTINEL} (plus negative values for {BINARY_RAW})")
```



```
# ---- Data quality checks ----
HIDDEN = SENTINEL + [c for c in ["intended_balcon_amount"] if c in df
def is_missing(frame, c):
    """BAF stores 'missing' as -1 (or a negative number in intended_b
    return frame[c] < 0 if c == "intended_balcon_amount" else frame[c

miss = pd.DataFrame({"hidden_missing_pct": {c: is_missing(df, c).mean
display(miss.round(2))
n_nan, n_dup = int(df.isna().sum().sum()), int(df.duplicated().sum())
const_cols = [c for c in df.columns if df[c].nunique() <= 1]
checks = {"income outside [0, 1]": ((df["income"] < 0) | (df["income"
        "name_email_similarity outside [0, 1]": ((df["name_email_si
        "negative days_since_request": (df["days_since_request"] < 1
        "negative velocity values": (df[["velocity_6h", "velocity_2

# ---- Cleaning ----
df_model = df.drop_duplicates().reset_index(drop=True)
df_plot = df_model.copy()
for c in HIDDEN: df_plot[c] = df_plot[c].where(~is_missing(df_plot, c

interpret(f"Most missing column: **{miss.index[0]}** ({miss['hidden_m
        f"Normal NaN cells: {n_nan}. Duplicate rows removed: {n_dup
        f"Impossible values: {(k: int(v) for k, v in checks.items(
        f"Clean table: **{len(df_model):,} rows**. Outliers (burst
```

	hidden_missing_pct 
intended_balcon_amount	74.3300
prev_address_months_count	71.2500
bank_months_count	25.4100
current_address_months_count	0.4300
session_length_in_minutes	0.2000
device_distinct_emails_8w	0.0400

Interpretation

- Most missing column: **intended_balcon_amount** (74.3% of rows hold -1 or a negative value). A scaler or model would read -1 as a real number, so we convert it to NaN and keep an explicit missing flag.
- Normal NaN cells: 0. Duplicate rows removed: 0. Constant columns (no information, dropped later): ['device_fraud_count'].
- Impossible values: {'income outside [0, 1]': 0, 'name_email_similarity outside [0, 1]': 0,

✓ 1.1 Class imbalance and drift over time

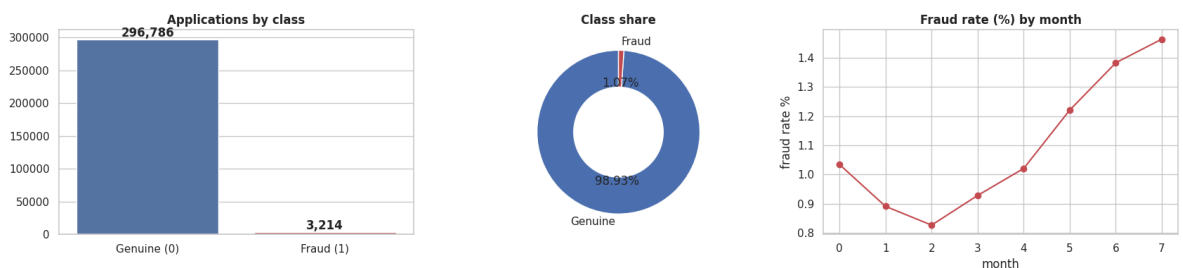
Fraud is rare. If only a small share of applications are fraud, a model that always says "genuine" looks very accurate and catches nothing. We measure the imbalance first, and also check whether the fraud rate changes from month to month (this decides how we split the data).

```

counts = df_model[TARGET].value_counts().sort_index()
n_pos_all, n_neg_all = int(counts.get(1, 0)), int(counts.get(0, 0))
fraud_rate = df_model[TARGET].mean(); imbalance_ratio = n_neg_all / n_pos_all
by_month = df_model.groupby("month")[TARGET].agg(rate="mean", n_fraud="sum")

fig, ax = plt.subplots(1, 3, figsize=(17, 4))
sns.barplot(x=["Genuine (0)", "Fraud (1)"], y=counts.values, palette=
for i, v in enumerate(counts.values): ax[0].text(i, v, f"{v:,}", ha="
ax[0].set_title("Applications by class")
ax[1].pie(counts.values, labels=["Genuine", "Fraud"], colors=[BLUE, RED])
ax[1].set_title("Class share")
ax[2].plot(by_month.index, by_month["rate"], marker="o", color=RED);
plt.tight_layout(); plt.show()
interpret(f"{n_neg_all:,} genuine vs {n_pos_all:,} fraud applications
f"Always predicting 'genuine' would be {100 - fraud_rate * 100}% accurate"
f"Fraud rate by month goes from {by_month['rate'].min():.2f}% to {by_month['rate'].max():.2f}%")

```



Interpretation

- 296,786 genuine vs 3,214 fraud applications: fraud is **1.07%** of the data (about 92 genuine for every fraud).
- Always predicting 'genuine' would be 98.93% accurate and catch **zero** fraud, so we judge models with **recall, precision, F1, ROC-AUC and PR-AUC**, not accuracy.
- Fraud rate by month goes from 0.83% to 1.46%. Fraud behaviour changes over time, so we train on the past and test on the future (a time split) instead of shuffling randomly.

✓ 1.2 Missing fields and flags as fraud signals

Genuine customers usually have a history (previous address, previous bank account, device history). If that history is **missing**, the application may be riskier. We compare the fraud rate when a field is missing with when it is present, and the fraud rate for each 0/1 flag.

```

rows = []
for c in HIDDEN:

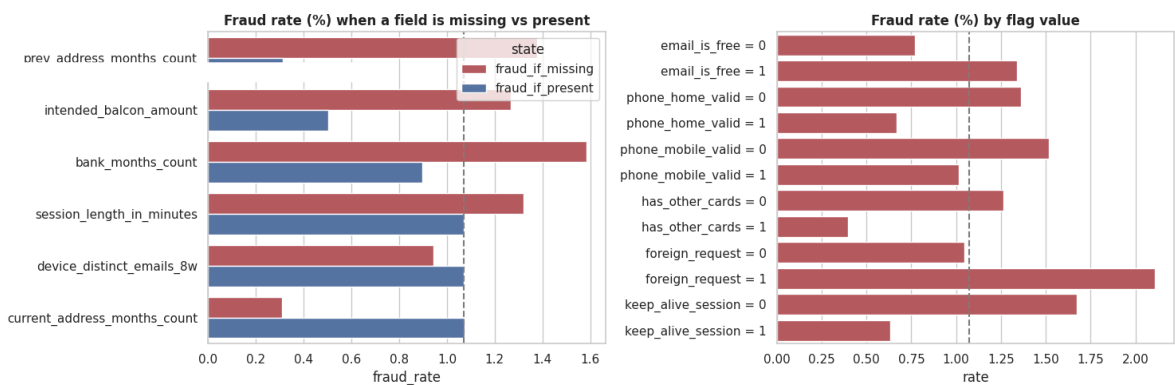
```

```

m = is_missing(df_model, c)
if 0 < m.sum() < len(m): rows.append((c, m.mean() * 100, df_model
gap = pd.DataFrame(rows, columns=["column", "pct_missing", "fraud_if_
gap["gap"] = gap["fraud_if_missing"] - gap["fraud_if_present"]; gap =

fig, ax = plt.subplots(1, 2, figsize=(15, 5))
g = gap.melt(id_vars="column", value_vars=["fraud_if_missing", "fraud
sns.barplot(data=g, y="column", x="fraud_rate", hue="state", palette=
ax[0].set_title("Fraud rate (%) when a field is missing vs present");
fl = pd.DataFrame([(f"{c} = {int(v)}", r * 100) for c in BINARY_RAW f
sns.barplot(data=fl, y="label", x="rate", color=RED, ax=ax[1]); ax[1]
ax[1].set_title("Fraud rate (%) by flag value"); ax[1].set_ylabel("")
plt.tight_layout(); plt.show()
top = gap.iloc[0]; fl_top = fl.sort_values("rate", ascending=False).i
interpret(f"Biggest gap: **{top['column']}** has fraud rate {top['fra
f"Highest-risk flag value: **{fl_top['label']}** with fraud
"Where the red 'missing' bar is far above the blue one, *be

```



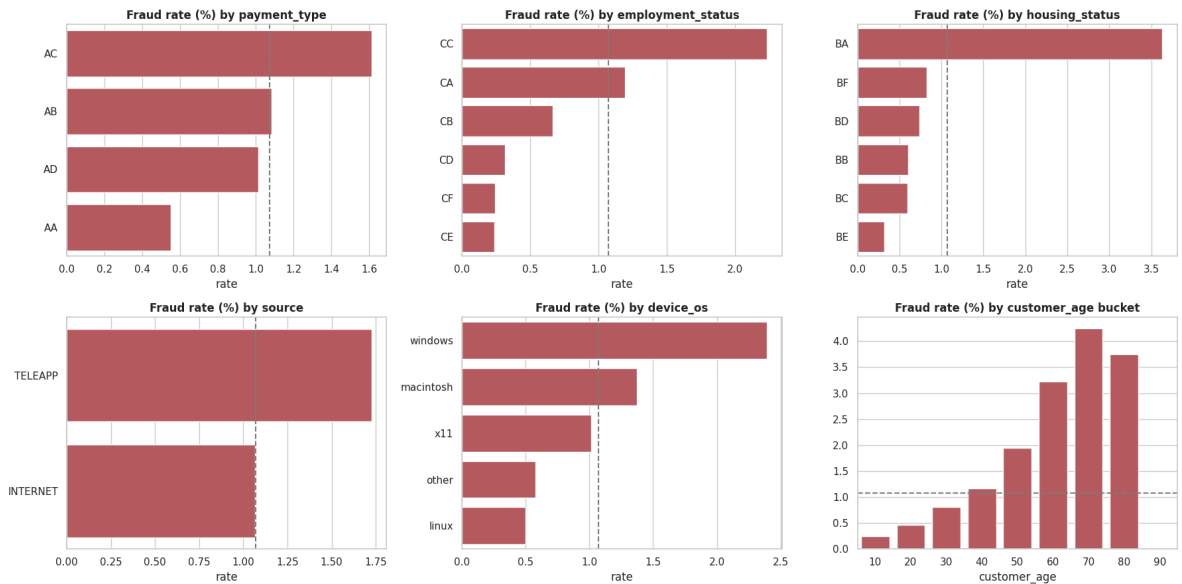
Interpretation

- Biggest gap: **prev_address_months_count** has fraud rate 1.38% when missing vs 0.31% when present (dashed line = overall 1.07%).
- Highest-risk flag value: **foreign_request = 1** with fraud rate 2.11%.
- Where the red 'missing' bar is far above the blue one, *being missing* is itself a signal, so we create a missing flag for each of these columns in feature engineering.

✓ 1.3 Categories and age

We check which category values and which age groups have more fraud (only groups with at least 200 applications, since tiny groups give unreliable rates).

```
cat_show = CAT_RAW[:5]
fig, axes = plt.subplots(2, 3, figsize=(18, 9)); axes = axes.ravel()
best_cat = []
for ax, c in zip(axes, cat_show):
    t = df_model.groupby(c)[TARGET].agg(count="size", rate="mean").reset_index()
    t = t[t["count"] >= 200].sort_values("rate", ascending=False); t['rate'] = t['rate'] * 100
    sns.barplot(data=t, y=c, x="rate", color=RED, ax=ax); ax.axvline(x=fraud_rate * 100)
    if len(t): best_cat.append(f"`{c}`: **{t.iloc[0][c]}** at {t.iloc[0][c]}%")
age_t = df_model.groupby("customer_age")[TARGET].mean() * 100
sns.barplot(x=age_t.index.astype(int).astype(str), y=age_t.values, color=RED, ax=axes[-1]);
axes[-1].axhline(fraud_rate * 100, ls="--", c="grey"); axes[-1].set_xlabel("Age")
for ax in axes[1:-1]: ax.axis("off")
plt.tight_layout(); plt.show()
interpret("Highest fraud rate among frequent groups: " + "; ".join(best_cat) + "\n" +
f"Age bucket with the highest fraud rate: **{int(age_t.idxmax())}**\n" +
"Age is a sensitive attribute. A model may use it as a short-cut")
```



Interpretation

- Highest fraud rate among frequent groups: payment_type: **AC** at 1.6% (n=75,735); employment_status: **CC** at 2.2% (n=11,252); housing_status: **BA** at 3.6% (n=50,852); source: **TELEAPP** at 1.7% (n=2,087); device_os: **windows** at 2.4% (n=78,651).
- Age bucket with the highest fraud rate: **70** (4.25%), lowest: **90** (0.00%).
- Age is a sensitive attribute. A model may use it as a shortcut and treat older or younger genuine customers unfairly, so we run a fairness check in Phase 7.

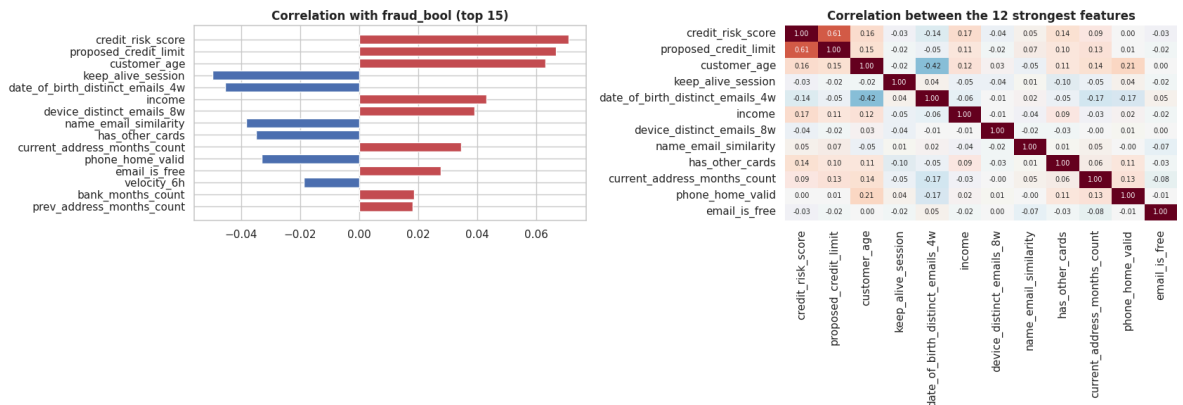
✓ 1.4 Correlation

Correlation shows linear links only, so a weak value does not make a feature useless. In fraud data the signal is usually spread over many weak, non-linear patterns. The second chart shows features that say almost the same thing (redundant).

```

corr = df_plot[NUM_RAW + [TARGET]].corr()
tcorr = corr[TARGET].drop(TARGET).dropna().sort_values(key=np.abs, as
top12 = tcorr.head(12).index.tolist()
fig, ax = plt.subplots(1, 2, figsize=(17, 6))
t15 = tcorr.head(15)[::-1]; ax[0].barh(t15.index, t15.values, color=[l
sns.heatmap(corr.loc[top12, top12], cmap="RdBu_r", center=0, annot=Tr
plt.tight_layout(); plt.show()
cm = corr.drop(index=TARGET, columns=TARGET).abs()
pairs = cm.where(np.triu(np.ones(cm.shape), k=1).astype(bool)).stack(
interpret(f"Strongest linear link with the target: **{tcorr.index[0]}":
"Redundant pairs (|r| > 0.7): " + ("", ".join(f"{a} and {b}

```



Interpretation

- Strongest linear link with the target: **credit_risk_score** (0.071). Values this small are normal in fraud data.
- Redundant pairs ($|r| > 0.7$): none. This hurts Logistic Regression a little but not tree models.

Phase 2: Feature Engineering, Preprocessing and Split

Leakage rule. Every engineered feature is calculated **row by row** from that application's own fields, with no use of the target and no statistics from other rows.

Anything that has to *learn* from data (median for filling gaps, scaling, one-hot categories) is fitted on the **training months only**.

Feature	Why it can help
<column>_missing flags, n_missing_flags	Missing history can itself signal fraud
credit_limit_to_income	A very high credit limit compared with income is risky
velocity_6h_to_24h, velocity_24h_to_4w	Shows a sudden burst of applications (acceleration)
address_months_total, address_to_age	Living-stability of the applicant
phones_valid	How many phone numbers could be verified
low_similarity_free_email	Free email address that does not look like the applicant's name

Why a time split. A random split would mix months, so the model would be tested on the same period it learned from. Fraud changes over time, so we train on months 0 to 4, tune on month 5 and test once on months 6 and 7, like a real bank would (learn from the past, face the future). `month` itself is never a feature.

```
FLAG_COLS = [c + "_missing" for c in HIDDEN]
NEW_FEATS = ["credit_limit_to_income", "velocity_6h_to_24h", "velocity_24h_to_4w",
             "phones_valid", "low_similarity_free_email", "n_missing_flags"]

def prepare(d):
    """Row-by-row cleaning and features only, so it is safe to run before model training"""
    d = d.copy()
    for c in HIDDEN:
        miss_c = is_missing(d, c)
        d[c + "_missing"] = miss_c.astype("int8")
        d[c] = d[c].where(~miss_c)
    d["credit_limit_to_income"] = d["proposed_credit_limit"] / (d["current_credit_limit"] + 1)
    d["velocity_6h_to_24h"] = d["velocity_6h"] / (d["velocity_24h"] + 1)
    d["velocity_24h_to_4w"] = d["velocity_24h"] / (d["velocity_4w"] + 1)
    d["address_months_total"] = d["current_address_months_count"]
    d["address_to_age"] = d["current_address_months_count"]
    d["phones_valid"] = d["phone_home_valid"] + d["phone_mobile_valid"]
    d["low_similarity_free_email"] = ((d["name_email_similarity"] < 0.5) && (d["name_email_similarity"] > 0.2))
    d["n_missing_flags"] = d[FLAG_COLS].sum(axis=1)
    return d.drop(columns=[c for c in const_cols if c in d.columns])

df_p = prepare(df_model)
FEATURES = [c for c in df_p.columns if c not in [TARGET, "month"]]
CAT_FEATURES = [c for c in FEATURES if not pd.api.types.is_numeric_dtype(df_p[c])]
NUM_FEATURES = [c for c in FEATURES if c not in CAT_FEATURES]

# Do the new features carry signal? (how well one feature alone ranks)
sig = pd.Series({c: roc_auc_score(df_p[TARGET], df_p[c].fillna(df_p[c].mean())) for c in FEATURES})
sig = sig.reindex((sig - 0.5).abs().sort_values(ascending=False).index)
display(sig.head(8).to_frame())
interpret(f"{len(FEATURES)} candidate features: {len(NUM_FEATURES)} numeric features"
         f"Strongest new feature on its own: **{sig.index[0]}** (AUC {sig.index[0]:.3f})")
```

single_feature_AUC



n_missing_flags	0.6545
prev_address_months_count_missing	0.6030
phones_valid	0.4052
address_months_total	0.5941
address_to_age	0.5891
intended_balcon_amount_missing	0.5691
low_similarity_free_email	0.5636
bank_months_count_missing	0.5615

Interpretation

- 43 candidate features: 38 numeric (including flags) and 5 categorical. `fraud_bool` and `month` are excluded.
- Strongest new feature on its own: **n_missing_flags** (AUC 0.654; 0.5 would mean no signal, below 0.5 means it points the other way). Whether the new features really help is

```
def make_preprocessor():
    """Fresh, unfitted preprocessing: fill gaps with the median and s
    return ColumnTransformer([("num", Pipeline([("imp", SimpleImputer
        ("cat", OneHotEncoder(handle_unknown="i

tr, va, te = [df_p["month"].isin(m) for m in (TRAIN_MONTHS, VAL_MONTHS)]
y_train, y_val, y_test = [df_p.loc[m, TARGET].values.astype(int) for m in (tr, va, te)]
prep = make_preprocessor()
X_train = prep.fit_transform(df_p.loc[tr, FEATURES]).astype("float32")
X_val, X_test = [prep.transform(df_p.loc[m, FEATURES]).astype("float32") for m in (va, te)]
FEAT_NAMES = [n.split("__", 1)[1] for n in prep.get_feature_names_out()]
n_pos, n_neg = int(y_train.sum()), int((y_train == 0).sum()); SPW = n_pos / n_neg

split_tbl = pd.DataFrame({"Rows": [len(y_train), len(y_val), len(y_test)],
                             "Genuine": [(y_train == 0).sum(), (y_val == 0).sum(), (y_test == 0).sum()],
                             "Fraud rate %": [y_train.mean() * 100, y_val.mean() * 100, y_test.mean() * 100],
                             index=["Train (months 0-4)", "Validation (months 5-9)", "Test (months 10-12)"]})
display(split_tbl.round(3))
BASELINE_ACC = accuracy_score(y_test, np.zeros_like(y_test))
interpret(f"Train: {len(y_train):,} applications ({n_pos:,} fraud). Validation: {len(y_val):,} applications ({n_pos + n_neg:,} fraud). V
    f"The 'always genuine' baseline reaches {BASELINE_ACC * 100}% accuracy.
    "Validation is used to tune models and to choose thresholds")
```

	Rows	Fraud	Genuine	Fraud rate %
Train (months 0-4)	202559	1901	200658	0.9380
Validation (month 5)	35881	438	35443	1.2210
Test (months 6-7)	61560	875	60685	1.4210

Interpretation

- Train: 202,559 applications (1,901 fraud). Validation: 35,881 (438 fraud). Test: 61,560 (875 fraud). With so few fraud cases in the test months, one extra miss moves recall by about 0.11 points, so read small differences with care.
- The 'always genuine' baseline reaches 98.58% accuracy with recall 0. After preprocessing (fitted on train only) the model sees 64 columns.

```
# ---- Evaluation helpers used for the rest of the notebook ----
def tpr_at_fpr(y, p, target=0.05):
    """Share of fraud caught if at most `target` (5%) of genuine appl
    fpr, tpr, _ = roc_curve(y, p); return float(tpr[fpr <= target].ma
def thr_at_fpr(y, p, target=0.05):
    fpr, tpr, thr = roc_curve(y, p); return float(thr[np.where(fpr <=
def thr_best_f1(y, p):
    pr, rc, thr = precision_recall_curve(y, p); f1 = 2 * pr * rc / (p
def confusion(y, prob, thr):
    tn, fp, fn, tp = confusion_matrix(y, (np.asarray(prob) >= thr).as
def compute_metrics(y, prob, thr):
    pred = (np.asarray(prob) >= thr).astype(int); tn, fp, fn, tp = co
    return {"Accuracy": accuracy_score(y, pred), "Precision": precisi
            "F1": f1_score(y, pred, zero_division=0), "ROC_AUC": roc_
            "TPR@5%FPR": tpr_at_fpr(y, prob), "TP": tp, "FP": fp, "FN"

RESULTS = {}
def add_result(name, kind, val_prob, test_prob, est=None):
    """Store scores and test metrics. The 0/1 threshold is chosen on
    thr = thr_best_f1(y_val, val_prob)
    m = compute_metrics(y_test, test_prob, thr); m["Threshold"] = thr
    RESULTS[name] = {"kind": kind, "val": np.asarray(val_prob).ravel(
    print(f"{name:<20} Prec={m['Precision']:.4f} Rec={m['Recall']:.4f
```

✓ Phase 3: Handling Class Imbalance

We compare five ways to deal with the imbalance using the same fast LightGBM model: no handling, **class weights**, random undersampling, random oversampling and **SMOTE** (synthetic fraud examples). Every resampling step is fitted on the **training data only**, never on validation or test. The comparison is scored on the **validation month**, so the test months stay untouched.

```
def quick_lgbm(w=1.0):
    return LGBMClassifier(n_estimators=100, learning_rate=0.1, num_le
    strategies = {"No handling": (None, 1.0), "Class weight": (None, SPW)
```

```

    "Undersampling (1:10)": (RandomUnderSampler(sampling_st
    "Oversampling (1:10)": (RandomOverSampler(sampling_stra
    "SMOTE (1:10)": (SMOTE(sampling_strategy=0.1, random_st

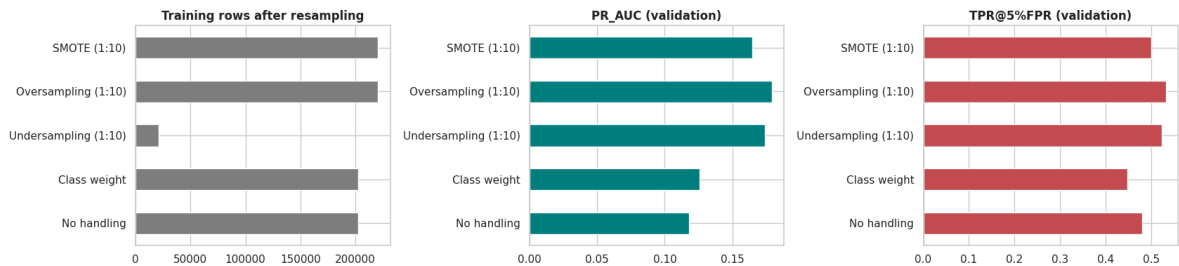
rows = []
for name, (sampler, w) in strategies.items():
    Xr, yr = (X_train, y_train) if sampler is None else sampler.fit_r
    pv = quick_lgbm(w).fit(Xr, yr).predict_proba(X_val)[: , 1]
    rows.append({"Strategy": name, "train_rows": len(yr), "PR_AUC": a
imb = pd.DataFrame(rows).set_index("Strategy"); display(imb.round(4))
fig, ax = plt.subplots(1, 3, figsize=(17, 4))
for a, (col, color) in zip(ax, [("train_rows", "grey"), ("PR_AUC", "t
plt.tight_layout(); plt.show()
# ---- DECISION: the strategy with the best VALIDATION PR-AUC is used
best_imb = imb["PR_AUC"].idxmax()
sampler_best, W_FIT = strategies[best_imb] # W_FIT =
X_fit, y_fit = (X_train, y_train) if sampler_best is None else sample
CLASS_W = {0: 1.0, 1: float(W_FIT)} if W_FIT > 1 else None # used b
MLP_W = float(np.sqrt(W_FIT)) if W_FIT > 1 else 1.0 # mild
imb_note = f"'{best_imb}' had the best validation PR-AUC ({imb['PR_AU
interpret(f"Best validation PR-AUC: **{best_imb}** ({imb['PR_AUC'].ma
f"Decision: all models below are trained on the data produc
"Undersampling throws most genuine rows away, and oversampl

```

train_rows PR_AUC ROC_AUC TPR@5%FPR 

Strategy

No handling	202559	0.1180	0.8722	0.4795
Class weight	202559	0.1260	0.8345	0.4475
Undersampling (1:10)	20911	0.1741	0.8779	0.5228
Oversampling (1:10)	220723	0.1792	0.8782	0.5320
SMOTE (1:10)	220723	0.1648	0.8785	0.5000



Interpretation

- Best validation PR-AUC: **Oversampling (1:10)** (0.1792). Class weights reach 0.1260; SMOTE reaches 0.1648; no handling reaches 0.1180.
- Decision: all models below are trained on the data produced by **Oversampling (1:10)** (220,723 training rows, 20,065 fraud rows). The choice was made on the validation month, so the test months stay untouched.
- Undersampling throws most genuine rows away, and oversampling or SMOTE adds copies or synthetic points. Resampling is applied to the TRAINING months only; validation and test keep their real, natural fraud rate.

✓ Phase 4: Machine Learning Track

We compare three models, all trained on the training months only:

1. **Logistic Regression**, the simple, explainable baseline.
2. **Random Forest**, many decision trees voting together.
3. **LightGBM**, gradient boosting (each new tree fixes the errors of the earlier ones), very strong on tables like this one. We then **tune** it with `RandomizedSearchCV`, optimising **PR-AUC** (not accuracy), and compare it before and after tuning.

Every model is scored on the same untouched test months. The 0/1 threshold of each model is picked on the validation month (the value that gives the best F1). Section 6 then looks at better thresholds.

```
models = {"Logistic Regression": LogisticRegression(C=0.1, class_weight=
          "Random Forest": RandomForestClassifier(n_estimators=N_TREE
          class_weight=CLASS_
          "LightGBM": LGBMClassifier(n_estimators=N_TREES, learning_r
          colsample_bytree=0.8, reg_lambda=
for name, est in models.items():
    est.fit(X_fit, y_fit) # trained
    add_result(name, "ML", est.predict_proba(X_val)[: , 1], est.predic
b = pd.DataFrame({n: RESULTS[n]["metrics"] for n in RESULTS}).T.astype
interpret(f"Accuracy is not shown here on purpose: the 'always genuin
f"Best baseline by PR-AUC: **{b['PR_AUC'].idxmax()}**. Prec
```

Logistic Regression	Prec=0.2149	Rec=0.2377	F1=0.2257	ROC-AUC=0.8803	P
Random Forest	Prec=0.1807	Rec=0.3863	F1=0.2462	ROC-AUC=0.8732	P
LightGBM	Prec=0.2368	Rec=0.2823	F1=0.2576	ROC-AUC=0.8848	P

Interpretation

- Accuracy is not shown here on purpose: the 'always genuine' baseline already gets 0.986. Recall ranges from 0.238 to 0.386 and PR-AUC from 0.157 to 0.180 (a model with no skill would get a PR-AUC of about 0.014).
- Best baseline by PR-AUC: **LightGBM**. Precision is low for every model because even a

```
# Tune LightGBM: random search where every candidate trains on the TR
X_tv, y_tv = np.vstack([X_fit, X_val]), np.concatenate([y_fit, y_val]
cv_split = PredefinedSplit(np.r_[np.full(len(y_fit), -1), np.zeros(len
space = {"n_estimators": [200, 300, 450], "num_leaves": [15, 31, 63],
        "subsample": [0.6, 0.8, 1.0], "colsample_bytree": [0.5, 0.7,
search = RandomizedSearchCV(LGBMClassifier(subsample_freq=1, random_s
        scoring={"pr_auc": "average_precision", "
t0 = time.time(); search.fit(X_tv, y_tv)
bi = int(np.argmax(search.cv_results_["rank_test_pr_auc"])); best_par
tuned = LGBMClassifier(subsample_freq=1, random_state=SEED, n_jobs=-1
add_result("LightGBM (Tuned)", "ML", tuned.predict_proba(X_val)[: , 1]
before, after = RESULTS["LightGBM"]["metrics"], RESULTS["LightGBM (Tu
print("Best parameters:", {k: (round(float(v), 4) if isinstance(v, (f
d = after["PR_AUC"] - before["PR_AUC"]
interpret(f"Validation-month results of the best setting: PR-AUC {sea
f"Test PR-AUC moved from {before['PR_AUC']:.4f} to {after['
"Tuning does not always help. A setting that is best on mon
```

LightGBM (Tuned)	Prec=0.2520	Rec=0.2514	F1=0.2517	ROC-AUC=0.8901	P
Best parameters: {'subsample': 1.0, 'scale_pos_weight': 2.0, 'reg_lambda': 0.001}					

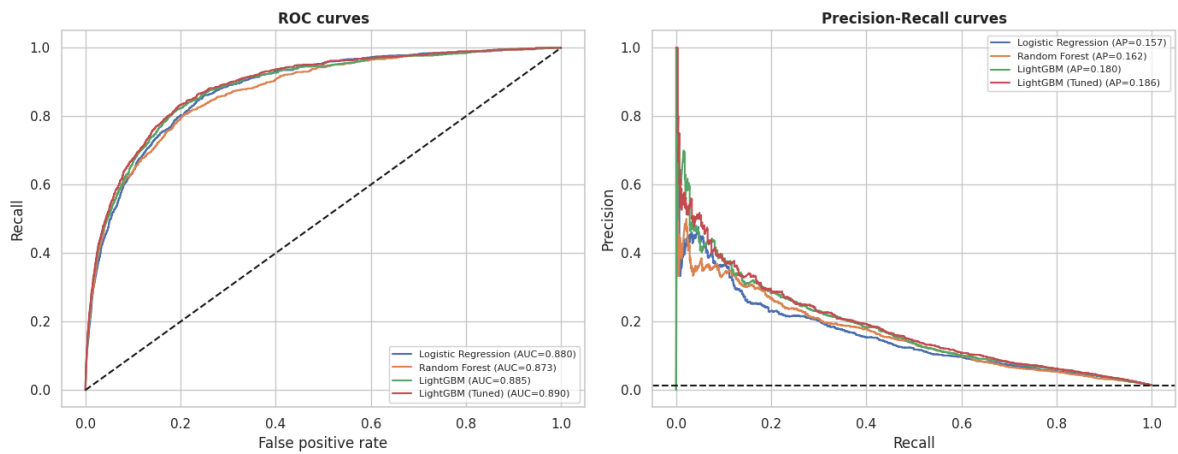
Interpretation

- Validation-month results of the best setting: PR-AUC 0.1817, recall 0.3425, precision 0.1818.
- Test PR-AUC moved from 0.1800 to 0.1860 (+0.0060); recall from 0.282 to 0.251. Tuning helped.
- Tuning does not always help. A setting that is best on month 5 may not be best on months 6 and 7 because fraud patterns drift and with four fraud cases small changes

✓ 4.1 ROC and Precision-Recall curves

The ROC curve is better when it hugs the top-left corner. The **Precision-Recall curve** is the stricter test for rare fraud: the dashed line shows what a random model would get (it equals the fraud rate). The huge number of genuine applications makes even a weak model look good on ROC, so the PR curve is the more honest of the two.

```
def plot_curves(names, title=""):
    fig, ax = plt.subplots(1, 2, figsize=(14, 5.5))
    for n in names:
        f, t, _ = roc_curve(y_test, RESULTS[n]["test"]); ax[0].plot(f,
            t, _ = precision_recall_curve(y_test, RESULTS[n]["test"]);
        ax[0].plot([0, 1], [0, 1], "k--"); ax[0].set(title="ROC curves" +
        ax[1].axhline(y_test.mean(), c="k", ls="--"); ax[1].set(title="Pr
    plt.tight_layout(); plt.show()
plot_curves(list(RESULTS))
interpret("A curve that stays above the others over the whole range i
        f"The random-model line in the PR plot sits at {y_test.mean
```



Interpretation

- A curve that stays above the others over the whole range is the better ranker; crossing curves mean the best model depends on how many alerts the bank can handle.
- The random-model line in the PR plot sits at 0.014. Curves far above it show real learning. Most of the difference between models appears at the left of the PR plot, where the very riskiest applications are.

Phase 5: Deep Learning Track (Keras Neural Network)

A neural network needs scaled, numeric input. Our preprocessing already gives that (numbers scaled, categories one-hot encoded, gaps filled), and the validation month is kept for early stopping, so no extra split is needed. The test months stay untouched.

Architecture (chosen for this data: many rows but very few fraud cases and only a few dozen inputs, so a small and regularised network; the exact row, fraud and input counts are printed by the cells above and below).

Part	Choice
Input	One neuron per column Takes the preprocessed features

Part	Choice	
Hidden layers	128, 64, 32 neurons, ReLU	Learns feature combinations whi
Batch normalisation	After every hidden layer	Stabilises training with mixed bin
Dropout	0.3, 0.3, 0.2	Randomly switches neurons off s
Output	1 neuron, sigmoid	Fraud probability between 0 and
Loss / optimizer	Binary cross-entropy / Adam (lr 0.001)	Standard yes-no loss; trained on
Batch size / epochs	1024 / up to EPOCHS	Large batches make sure every t
Callbacks	EarlyStopping on validation PR-AUC, ReduceLROnPlateau	Stop when learning stops helping

```
def build_mlp(input_dim):
    inp = keras.Input(shape=(input_dim,)); x = inp
    for units, drop in [(128, 0.3), (64, 0.3), (32, 0.2)]:
        x = layers.Dense(units, kernel_regularizer=regularizers.l2(1e-3))(x)
        x = layers.BatchNormalization()(x); x = layers.Activation("relu")(x)
    model = keras.Model(inp, layers.Dense(1, activation="sigmoid")(x))
    model.compile(optimizer=keras.optimizers.Adam(1e-3), loss="binary_crossentropy",
                  metrics=[keras.metrics.Precision(name="precision"),
                           keras.metrics.AUC(name="roc_auc"), keras.metrics.Recall(name="recall")])
    return model

keras.utils.set_random_seed(SEED)
mlp = build_mlp(X_fit.shape[1])
class_weight = {0: 1.0, 1: MLP_W}
print("Class weights:", class_weight, "| fraud rows in training data", X_fit.shape[0])
mlp.summary()
```

Class weights: {0: 1.0, 1: 1.0} | fraud rows in training data (after P
Model: "fraud_mlp"

Layer (type)	Output Shape	Par
input_layer (InputLayer)	(None, 64)	
dense (Dense)	(None, 128)	8
batch_normalization (BatchNormalization)	(None, 128)	
activation (Activation)	(None, 128)	
dropout (Dropout)	(None, 128)	
dense_1 (Dense)	(None, 64)	8
batch_normalization_1 (BatchNormalization)	(None, 64)	
activation_1 (Activation)	(None, 64)	
dropout_1 (Dropout)	(None, 64)	
dense_2 (Dense)	(None, 32)	2
batch_normalization_2 (BatchNormalization)	(None, 32)	
activation_2 (Activation)	(None, 32)	
dropout_2 (Dropout)	(None, 32)	
dense_3 (Dense)	(None, 1)	

Total params: 19,585 (76.50 KB)
Trainable params: 19,137 (74.75 KB)

```
cb = [callbacks.EarlyStopping(monitor="val_pr_auc", mode="max", patien
      callbacks.ReduceLROnPlateau(monitor="val_pr_auc", mode="max", f
history = mlp.fit(X_fit, y_fit, validation_data=(X_val, y_val), epoch
hist = pd.DataFrame(history.history); print(f"Stopped after {len(hist
```

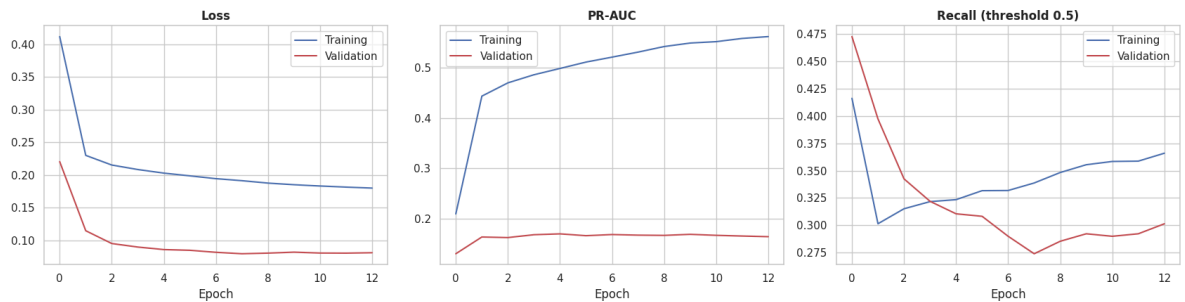
```
Epoch 1/40
216/216 - 7s - 32ms/step - loss: 0.4117 - pr_auc: 0.2091 - precision:
Epoch 2/40
216/216 - 3s - 12ms/step - loss: 0.2299 - pr_auc: 0.4433 - precision:
Epoch 3/40
216/216 - 2s - 12ms/step - loss: 0.2151 - pr_auc: 0.4697 - precision:
Epoch 4/40
216/216 - 3s - 12ms/step - loss: 0.2082 - pr_auc: 0.4858 - precision:
Epoch 5/40
216/216 - 3s - 14ms/step - loss: 0.2028 - pr_auc: 0.4984 - precision:
Epoch 6/40
216/216 - 3s - 13ms/step - loss: 0.1986 - pr_auc: 0.5110 - precision:
Epoch 7/40
216/216 - 3s - 12ms/step - loss: 0.1943 - pr_auc: 0.5209 - precision:
Epoch 8/40
```

```
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.000500000023748
216/216 - 2s - 11ms/step - loss: 0.1912 - pr_auc: 0.5309 - precision: 0.5309
Epoch 9/40
216/216 - 3s - 15ms/step - loss: 0.1875 - pr_auc: 0.5419 - precision: 0.5419
Epoch 10/40
216/216 - 3s - 13ms/step - loss: 0.1850 - pr_auc: 0.5489 - precision: 0.5489
Epoch 11/40

Epoch 11: ReduceLROnPlateau reducing learning rate to 0.000250000011875
216/216 - 2s - 11ms/step - loss: 0.1830 - pr_auc: 0.5517 - precision: 0.5517
Epoch 12/40
216/216 - 2s - 11ms/step - loss: 0.1813 - pr_auc: 0.5581 - precision: 0.5581
Epoch 13/40
216/216 - 2s - 11ms/step - loss: 0.1798 - pr_auc: 0.5618 - precision: 0.5618
Epoch 13: early stopping
Restoring model weights from the end of the best epoch: 5.
Stopped after 13 epochs
```

```
fig, ax = plt.subplots(1, 3, figsize=(17, 4.5))
for a, (m, t) in zip(ax, [("loss", "Loss"), ("pr_auc", "PR-AUC"), ("recall", "Recall")]):
    a.plot(hist[m], label="Training", c=BLUE); a.plot(hist[f"val_{m}"], label="Validation", c=RED)
plt.tight_layout(); plt.show()

dl_val = mlp.predict(X_val, batch_size=8192, verbose=0).ravel(); dl_train = mlp.predict(X_train, batch_size=8192, verbose=0).ravel()
DL_NAME = "Deep Learning (MLP)"; add_result(DL_NAME, "DL", dl_val, dl_train)
be = int(hist["val_pr_auc"].idxmax())
interpret(f"Best epoch by validation PR-AUC: {be + 1} (validation {hist[\"val_pr_auc\"][be]:.4f})")
print("**Overfitting check:** overfitting shows as training metrics increasing while validation metrics decrease")
```



Deep Learning (MLP) Prec=0.2308 Rec=0.2503 F1=0.2401 ROC-AUC=0.8762 P

Interpretation

- Best epoch by validation PR-AUC: 5 (validation 0.169, training 0.498, gap +0.329). At the last epoch, validation loss minus training loss is -0.099.
- **Overfitting check:** overfitting shows as training metrics that keep improving while validation metrics stall or get worse. Early stopping with restored best weights, dropout and L2 limit it. Part of the gap is also expected because the validation month is *later* than the training months (fraud drifts), and dropout is active only during training.

✓ Phase 6: Evaluation, Model Selection and Threshold

Reading the confusion matrix here. TP = fraud caught. TN = genuine application approved. FP = genuine customer wrongly flagged (delay, friction, analyst time). FN = fraud missed (direct financial loss). **False negatives are usually the costlier error**, so we pay close attention to recall while keeping the alert volume manageable.

We now put all models side by side. The **final model** is the one with the highest **PR-AUC on the validation month** (the best threshold-free measure for rare fraud). The test months are used only to *report* results, never to choose. We also show which model is cheapest under the assumed costs `COST_FN` and `COST_FP`, which are illustrative, not real bank numbers.

```
comparison = pd.DataFrame({n: r["metrics"] for n, r in RESULTS.items()})
comparison.insert(0, "Type", [r["kind"] for r in RESULTS.values()])
comparison["Expected_cost"] = comparison["FN"] * COST_FN + comparison["FP"] * COST_FP
NO_MODEL_COST = float(y_test.sum()) * COST_FN # approve
comparison["Val_PR_AUC"] = [average_precision_score(y_val, RESULTS[n]) for n in RESULTS]
comparison = comparison.sort_values("Val_PR_AUC", ascending=False)
show_cols = ["Type", "Val_PR_AUC", "Precision", "Recall", "F1", "ROC_AUC"]
```

```

display(comparison[show_cols].style.format({c: "{:.4f}" for c in ["Val", "Recall", "F1", "PR_AUC"]})
        .background_gradient(subset=["Recall", "F1", "PR_AUC"], cmap='magma'))

names = list(comparison.index)
fig, axes = plt.subplots(1, len(names), figsize=(4.1 * len(names), 3.1 * len(names)))
for a, n in zip(np.atleast_1d(axes), names):
    m = RESULTS[n]["metrics"]; sns.heatmap([m["TN"], m["FP"]], [m["FN"], m["TP"]],
        cbar_kws={"label": "Confusion Matrix", "format": "{:.4f}"},
        xticklabels=["Pred genuine", "Pred fraud"],
        yticklabels=["Actual genuine", "Actual fraud"])
plt.tight_layout(); plt.show()
plot_curves([DL_NAME] + [n for n in names if RESULTS[n]["kind"] == "ML"])

best_ml_name = comparison[comparison["Type"] == "ML"]["Val_PR_AUC"].idxmax()
FINAL_NAME = comparison["Val_PR_AUC"].idxmax()
best_business = comparison.sort_values(["Expected_cost", "PR_AUC"], ascending=[True, False])
fm, bm, dm = comparison.loc[FINAL_NAME], comparison.loc[best_ml_name], comparison.loc[best_business]
interpret(f"**Final model: {FINAL_NAME}** (highest validation PR-AUC: {bm['Val_PR_AUC']})\n"
        f"Best recall: {comparison['Recall'].idxmax()} ({comparison['Recall'].loc[comparison['Recall'].idxmax()]:.4f})\n"
        f"Best ML model (by validation): {best_ml_name} (test PR-AUC: {bm['Val_PR_AUC']})\n"
        "Close scores can swap places on another split because the metrics are noisy")

```


	Type	Val_PR_AUC	Precision	Recall	F1	ROC_AUC	PR_AUC
LightGBM (Tuned)	ML	0.1817	0.2520	0.2514	0.2517	0.8901	0.1860
LightGBM	ML	0.1768	0.2368	0.2823	0.2576	0.8848	0.1800

✓ 6.0 How ^{Deep} Sure are we? (uncertainty on the test months)
^{Learning} (MLP)

The test months hold only a few hundred fraud cases, so one PR-AUC number can be lucky or unlucky. We resample the test applications 200 times (bootstrap, with

replacement) and recompute PR-AUC each time. The spread of those values is the **95% range**. We use the same resamples for every model, so we can also ask: is the

final model's lead over each other model larger than the noise?



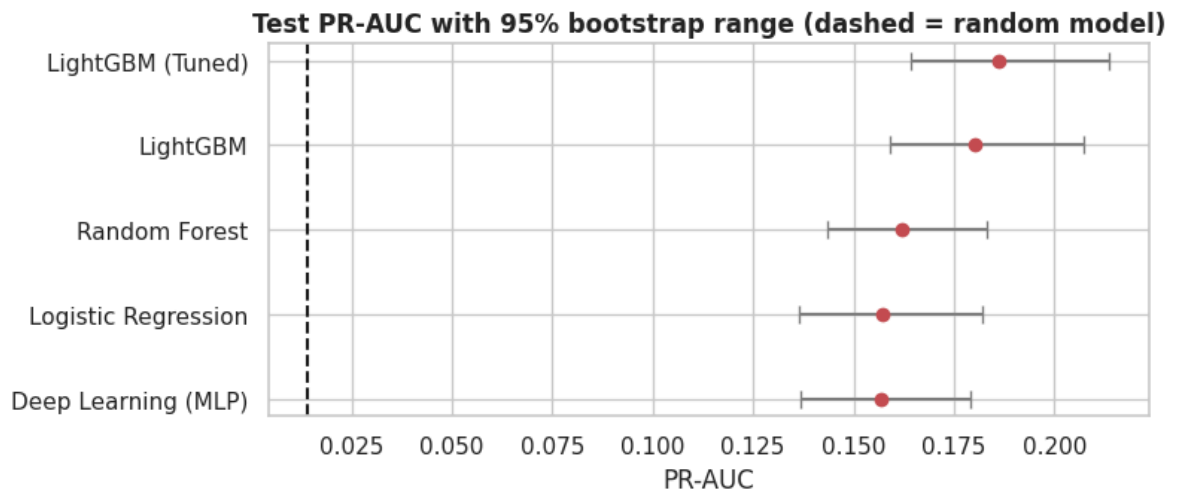
```
rng = np.random.RandomState(SEED); N_BOOT = 60 if FAST_MODE else 200
boot = {n: [] for n in RESULTS}
for _ in range(N_BOOT):
    i = rng.choice(len(y_test), len(y_test), replace=True) #
    if y_test[i].sum() == 0: continue
    for n in RESULTS: boot[n].append(average_precision_score(y_test[i
ci = pd.DataFrame({n: {"PR_AUC": RESULTS[n]["metrics"]["PR_AUC"], "lo
display(ci.round(4))

# Is the final model really better than each other model? (difference
diffs, verdicts = [], {}
for n in RESULTS:
    if n == FINAL_NAME: continue
    dd = np.array(boot[FINAL_NAME]) - np.array(boot[n]); lo, hi = np.
    verdicts[n] = "clearly better" if lo > 0 else "clearly worse" if l
    diffs.append((n, dd.mean(), lo, hi))

fig, ax = plt.subplots(figsize=(8, 3.6)); c2 = ci.iloc[:, -1]
ax.errorbar(c2["PR_AUC"], range(len(c2)), xerr=[c2["PR_AUC"] - c2["lo
ax.set_yticks(range(len(c2))); ax.set_yticklabels(c2.index); ax.axvli
plt.tight_layout(); plt.show()

n_sim = sum(v.startswith("similar") for v in verdicts.values())
CI_NOTE = (f"Over {N_BOOT} bootstrap resamples of the test months, {F
    f"Compared with the other {len(verdicts)} models it is " +
    + ("So the models are statistically very close, and the ho
interpret(f"{FINAL_NAME}: test PR-AUC {ci.loc[FINAL_NAME, 'PR_AUC']:.
    "Versus each other model: " + "; ".join(f"*{n}* {v}" for
    "If the ranges overlap and the differences are 'within noise'
```

	PR_AUC	low (2.5%)	high (97.5%)
LightGBM (Tuned)	0.1860	0.1643	0.2136
LightGBM	0.1800	0.1590	0.2074
Random Forest	0.1619	0.1434	0.1831
Logistic Regression	0.1572	0.1366	0.1821
Deep Learning (MLP)	0.1567	0.1369	0.1792



Interpretation

- LightGBM (Tuned): test PR-AUC 0.1860, 95% range 0.1643 to 0.2136.
- Versus each other model: **Logistic Regression** clearly better; **Random Forest** clearly better; **LightGBM** similar (difference within noise); **Deep Learning (MLP)** clearly better.
- If the ranges overlap and the differences are 'within noise', we cannot claim one model is truly better. We then prefer the one that is simpler to explain or cheaper to run.

✓ 6.1 Threshold optimisation

A model outputs a score, and the threshold decides when it becomes a "fraud" flag. A **lower threshold** catches more fraud (higher recall) but creates more false alarms; a **higher threshold** gives fewer false alarms but lets more fraud through. We recommend the threshold that lets **at most 5% of genuine applications** be flagged (a workload analysts can really handle); the cost-based threshold is shown only for comparison. We choose the threshold on the **validation month**, then apply it once to the test months. Choosing it on the test months would make the result look better than it is.

```
s_val, s_test = RESULTS[FINAL_NAME]["val"], RESULTS[FINAL_NAME]["test"]

def sweep(yt, p):
```

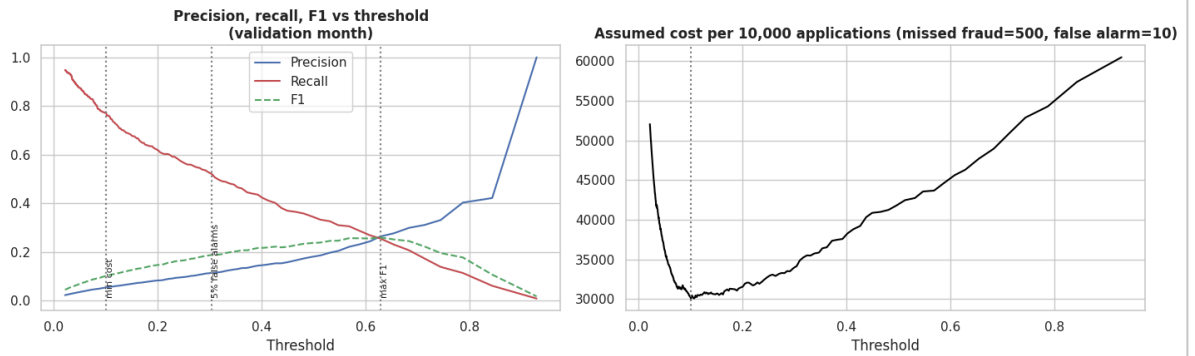
```

rows = []
for t in np.unique(np.quantile(p, np.linspace(0.5, 0.9999, 300)))
    tn, fp, fn, tp = confusion(yt, p, t)
    pr, rc = tp / max(tp + fp, 1), tp / max(tp + fn, 1)
    rows.append({"threshold": t, "precision": pr, "recall": rc, "
return pd.DataFrame(rows)
sw = sweep(y_val, s_val)
thr_f1 = float(sw.loc[sw["f1"].idxmax(), "threshold"])
thr_cost = float(np.median(sw.loc[sw["cost"] == sw["cost"].min(), "th
thr_fpr5, thr_fpr1 = thr_at_fpr(y_val, s_val, 0.05), thr_at_fpr(y_val
RECOMMENDED_THR = thr_fpr5 # realistic analyst workload: a

fig, ax = plt.subplots(1, 2, figsize=(14, 4.5))
ax[0].plot(sw["threshold"], sw["precision"], label="Precision", c=BLU)
for t, l in [(thr_f1, "max F1"), (thr_cost, "min cost"), (thr_fpr5, "
ax[0].set(title="Precision, recall, F1 vs threshold\n(validation mont
ax[1].plot(sw["threshold"], sw["cost"] / len(y_val) * 10000, c="black"
ax[1].set(title=f"Assumed cost per 10,000 applications (missed fraud=
plt.tight_layout(); plt.show()

ops = {"Default 0.5": 0.5, "Max F1": thr_f1, "Lowest assumed cost": t
tt = pd.DataFrame({lbl: {**compute_metrics(y_test, s_test, t), "Thres
tt["Alerts per 10k"] = (tt["TP"] + tt["FP"]) / len(y_test) * 10000; t
display(tt[["Threshold", "Precision", "Recall", "F1", "TP", "FP", "FN
rc_, d5 = tt.loc["5% false-alarm budget"], tt.loc["Default 0.5"]
interpret(f"Thresholds chosen on the validation month: max F1 {thr_f1
f"On the test months it gives recall {rc_['Recall']:.3f} an
"**Maximum fraud catch vs fewer false alarms:** a low thresh

```



	Threshold	Precision	Recall	F1	TP	FP	F
Default 0.5	0.5000	0.1986	0.3794	0.2607	332.0000	1,340.0000	543.0000
Max F1	0.6281	0.2516	0.2629	0.2571	230.0000	684.0000	645.0000
Lowest assumed cost	0.1014	0.0594	0.8149	0.1108	713.0000	11,283.0000	162.0000
5% false-alarm budget	0.3040	0.1264	0.5531	0.2057	484.0000	3,346.0000	391.0000
1% false-alarm budget	0.6162	0.2495	0.2709	0.2597	237.0000	713.0000	638.0000

Interpretation

- Thresholds chosen on the validation month: max F1 0.628, lowest assumed cost 0.101, 5% false-alarm budget 0.304, 1% budget 0.616. **Recommended: 0.304** (5% false-alarm budget: a workload analysts can handle; the lowest-assumed-cost threshold 0.101 is shown only for comparison).
- On the test months it gives recall 0.553 and precision 0.126, against recall 0.379 and precision 0.199 at 0.5, with about 622 alerts per 10,000 applications.
- Maximum fraud catch vs fewer false alarms:** a low threshold catches the most fraud but floods the analysts; a high one keeps the queue short but lets fraud through. The best point depends on the real cost ratio and on how many alerts the team can review, which is why the costs at the top are editable.

✓ Phase 7: Explainability and Business Interpretation

A fraud team must be able to explain **why** an application was flagged. For the best ML model we use two views: **built-in feature importance** and **permutation importance** (shuffle one column in the test data and measure how much PR-AUC drops, which works for any model). A third view, **SHAP**, shows the direction and size of each feature's effect; it is skipped safely if it fails in your environment.

```

est = RESULTS[best_ml_name]["estimator"]
native = pd.Series(est.feature_importances_ if hasattr(est, "feature_
native = (native / native.sum()).sort_values(ascending=False)
idx = np.random.RandomState(SEED).choice(len(y_test), size=min(40000,
perm = permutation_importance(est, X_test[idx], y_test[idx], scoring='
perm_df = pd.Series(perm.importances_mean, index=FEAT_NAMES).sort_val

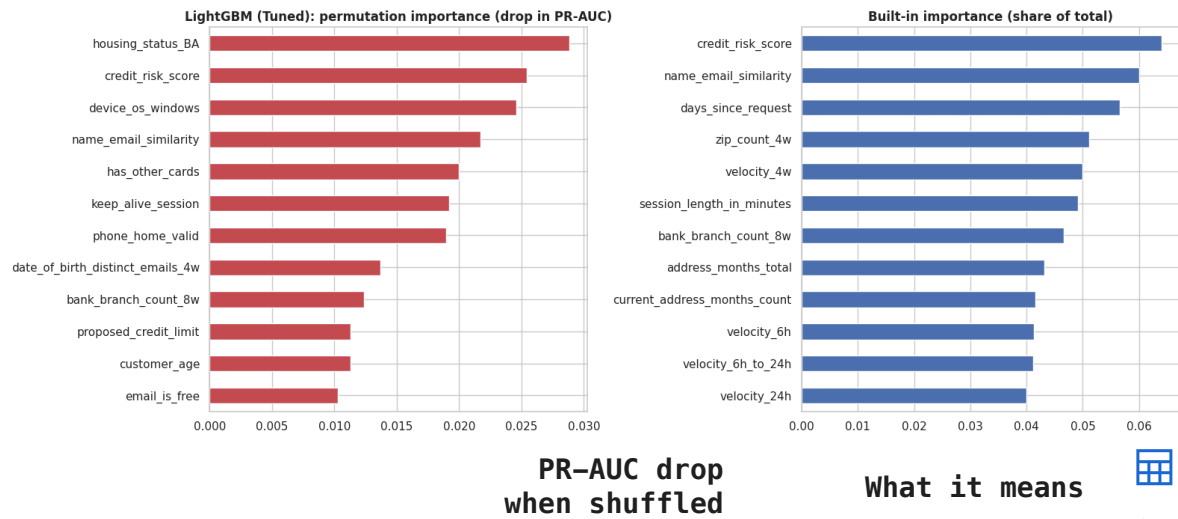
fig, ax = plt.subplots(1, 2, figsize=(16, 6))
perm_df.head(12)[::-1].plot(kind="barh", color=RED, ax=ax[0]); ax[0].
native.head(12)[::-1].plot(kind="barh", color=BLUE, ax=ax[1]); ax[1].
plt.tight_layout(); plt.show()

MEANING = {"name_email_similarity": "Email unlike the applicant's nam
"velocity_6h": "Burst of applications in 6 hours (automate
"velocity_6h_to_24h": "Acceleration of application bursts"
"zip_count_4w": "Many applications from one area recently"
"date_of_birth_distinct_emails_4w": "Same birth date used
"credit_risk_score": "The bank's own risk score of the app
"credit_limit_to_income": "Limit that is high compared with
"days_since_request": "Days since the application was made
"keep_alive_session": "'Keep me logged in' behaviour", "fo
"phone_home_valid": "Home phone could be verified", "phone
"current_address_months_count": "Time at current address",
"intended_balcon_amount": "Initial transfer amount", "n_mi
"address_to_age": "Time at address compared with age", "lo

def meaning(f):
    if f in MEANING: return MEANING[f]
    if f.endswith("_missing"): return f"History field '{f[:-8]}' is m
    for pre in CAT_RAW:
        if f.startswith(pre + "_"): return f"Category '{f[len(pre) +
    return ""

top_perm_names, native_top = perm_df.head(5).index.tolist(), native.h
tab = perm_df.head(8).to_frame("PR-AUC drop when shuffled"); tab["Wha
interpret(f"Most important columns by permutation: **{'', '.join(top_p
    f"Top built-in features: {'', '.join(native_top)).",
    f"Engineered features in the permutation top 8: {[f for f in
    "Values near zero mean the model barely uses that column. In

```



housing_status_BA	0.0289	Category 'BA' of housing_status (anonymised le...
credit_risk_score	0.0254	The bank's own risk score of the applicant
device_os_windows	0.0246	Category 'windows' of device_os (anonymised le...
name_email_similarity	0.0217	Email unlike the applicant's name (possible fa...
has_other_cards	0.0200	Existing relationship with the bank
keep_alive_session	0.0192	'Keep me logged in' behaviour
phone_home_valid	0.0190	Home phone could be verified
date_of_birth_distinct_emails_4w	0.0137	Same birth date used with many emails (identit...

Interpretation

- Most important columns by permutation: **housing_status_BA**, **credit_risk_score**, **device_os_windows**, **name_email_similarity**, **has_other_cards**. Shuffling housing_status_BA lowers PR-AUC by 0.0289, more than any other column.
- Top built-in features: credit_risk_score, name_email_similarity, days_since_request, zip_count_4w, velocity_4w, session_length_in_minutes.
- Engineered features in the permutation top 8: none.
- Values near zero mean the model barely uses that column. Importance shows association, not cause, and fraudsters change their behaviour over time.